



BASES DE DATOS

FACULTAD DE ESTUDIOS ESTADÍSTICOS

ADMINISTRACIÓN DE FINCAS

Pablo Galarón Mateo
Raúl Palomo Mazo

ÍNDICE

1. Introducción.....	2
2. Enunciado.....	3
3. Modelo Entidad - Relación.....	4
4. Modelo Relacional.....	6
5. Script SQL.....	7
6. Conclusiones.....	28

1. Introducción.

El trabajo se enfoca en el diseño y gestión de una base de datos para una empresa de administración de fincas que gestiona múltiples sedes en diversas ubicaciones. El objetivo principal es estructurar un sistema eficiente para administrar toda la información relacionada con las sedes, comunidades, portales, vecinos, instalaciones y empresas externas que colaboran con las comunidades. A través de esta base de datos, se busca optimizar la gestión de operaciones diarias, así como facilitar el acceso a información clave como puede ser los horarios de atención, los datos del personal, los elementos comunes en cada portal, las necesidades de mantenimiento o las reuniones comunitarias.

Para garantizar la integridad de los datos y su correcta manipulación, hemos creado un modelo relacional en SQL que incorpora diversas tablas interrelacionadas, junto con triggers que permiten la inserción de datos de manera coherente y el cumplimiento de las reglas de integridad. Además, se han desarrollado consultas SQL específicas para obtener información relevante y útil para la toma de decisiones dentro de la administración de fincas. Este enfoque permite no solo la gestión eficiente de recursos, sino también la mejora de la comunicación entre los diferentes actores involucrados, desde los vecinos hasta las empresas contratadas para la satisfacción de las necesidades que pueden surgir en las distintas comunidades.

En total hemos creado en MySQLWorkbench:

- 18 tablas
- 3 triggers de inserción
- Otros 2 triggers distintos
- 16 consultas

Si se ejecuta el script que está adjuntado justo con este pdf, se obtendrán todos los resultados. Se podrá ejecutar directamente todo el script y no dará ningún error. Recomendamos si se quiere ver algo de la implementación en sql, abrir el script ya que se ve de forma mucho más clara y ordenada que en este documento pdf.

2. Enunciado.

Una administradora de fincas gestiona múltiples **sedes** y actúa como un centro operativo. Cada sede se identifica por un código único y se desea conocer los horarios de atención y la dirección.

Cada sede tiene una ubicación específica dentro de una **ciudad** (código, nombre y población) y contrata **personal** administrativo encargado de la gestión local (es de interés conocer cuando esa persona es contratada). Dentro del personal, se designa un jefe que lidera la sede y supervisa sus operaciones. Del personal se desea conocer su DNI, nombre y apellidos, salario, especialidad, email y teléfonos. Cada trabajador vive en una ciudad y trabaja en una determinada sede.

Las sedes administran varias **comunidades** (código comunidad, nombre y estatutos) y una comunidad es administrada por una única sede. Cada una de las comunidades está formada por múltiples **portales**. Las comunidades tienen un presidente, que es elegido entre los vecinos cada 4 años (un vecino podrá ser elegido presidente en diferentes ocasiones). Cada portal tiene un número de viviendas, un número de portal y un código único. Además, en cada portal residen **vecinos**, quienes se identifican por su DNI, nombre, apellidos, número de cuenta, teléfonos y edad. Los vecinos pueden ser propietarios o arrendatarios y es de interés conocer desde cuando residen en la comunidad y, en el caso de que la abandonen, la fecha en la que lo hacen.

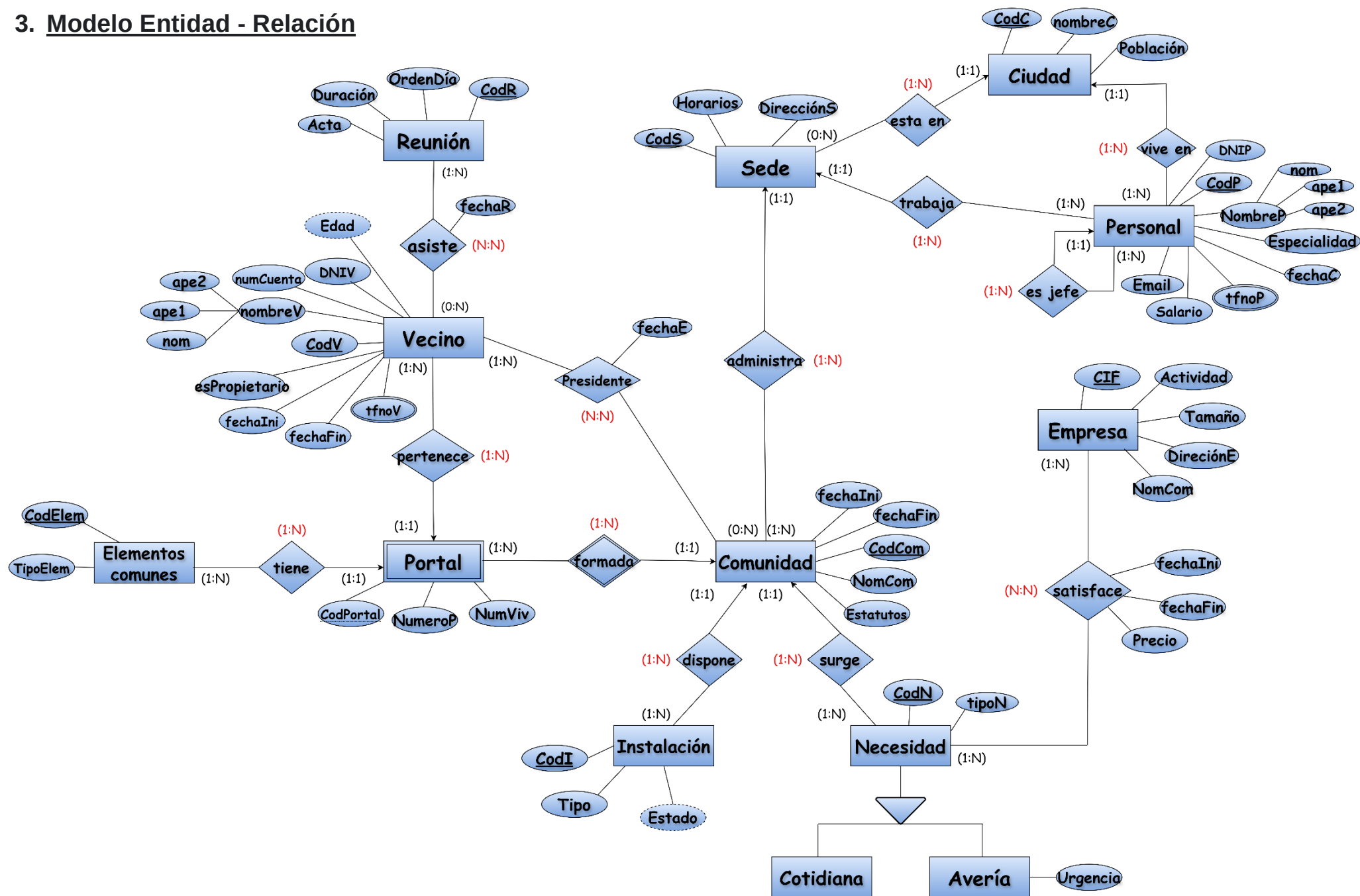
Cada portal tiene **elementos comunes** (código de elemento, tipo).

Dentro de las comunidades existen diversas **instalaciones** (piscinas, jardines, gimnasios etc.), que están asociadas a su comunidad. Las instalaciones tienen un estado que indica su condición actual. A su vez, las comunidades pueden tener **necesidades** específicas, que pueden clasificarse como **cotidianas**(limpieza, seguridad, etc.) o relacionadas con **averías**, que pueden ser más o menos urgentes.

Para cubrir las necesidades de las comunidades, las sedes contratan **empresas** externas. Las empresas están identificadas por su código único(CIF), nombre comercial, dirección, actividad principal y tamaño . Estas empresas ofrecen servicios específicos como limpieza, fontanería, mantenimiento de instalaciones, etc.

Las comunidades también celebran **reuniones** periódicas, en las que se tratan temas relevantes, como el estado de las instalaciones, la gestión de necesidades y el presupuesto comunitario. Cada reunión cuenta con un orden del día, una duración específica y un acta que recoge lo discutido. Solo los vecinos de una comunidad pueden asistir a las reuniones correspondientes a dicha comunidad

3. Modelo Entidad - Relación



El modelo entidad-relación presentado incluye diversas entidades, relaciones de cardinalidad 1:N, relaciones N:N, una relación reflexiva ("es jefe"), y una generalización (representada entre las necesidades "Cotidiana" y "Avería").

Es fundamental destacar que una vez diseñado el modelo entidad-relación, el siguiente paso clave es transformarlo en un modelo relacional para implementarlo después en sql. Debemos tener en cuenta lo siguiente:

- **Claves primarias:** Cada tabla debe tener una clave primaria que identifique de forma única cada fila en la tabla.
- **Claves ajenas :** Las claves ajenas deben estar claramente identificadas (**en rojo en este caso**) para establecer las relaciones entre tablas, conectándolas con las claves primarias correspondientes.
- **Claves alternativas:** Se representan en negrita. Son candidatas a ser claves primarias, pero sin serlo.
- **Valores nulos:** Es importante analizar cuáles atributos pueden aceptar valores nulos indicándolo con un *.
- **Atributos multivalorados:** Estos deben convertirse en tablas separadas que incluyan una clave primaria y una relación con la tabla original.
- **Relaciones 1:N:** En estas relaciones, el atributo clave del lado "1" pasa como clave ajena al lado "N".
- **Relaciones N:N:** Estas relaciones generan una tabla intermedia con claves ajenas provenientes de ambas entidades participantes. En algunos casos, dependiendo del contexto, puede ser necesario seleccionar varias claves primarias o incluir atributos adicionales en esta tabla para registrar información propia de la relación. Es decir, **hay MÁS DE 2 CLAVES PRIMARIAS DEPENDIENDO DEL CONTEXTO.**
- **Generalizaciones:** Se hacen tablas de las entidades por separado y la clave primaria será la de la entidad que está por encima.

4. Modelo Relacional (paso a tablas).

Sede (CodS, Horarios, DirecciónS, **CodC**)

Ciudad (CodC, nombreC, Población)

Personal (CodP, **DNIP**, nom, ape1, ape2(*), Especialidad, fechaC, Email, Salario, CodP, **CodC**, **CodS**)

tfnoP (numtelP, **CodP**)

Comunidad (CodCom, Estatutos, NomCom, fechaIni, fechaFin(*), **CodS**)

Portal (**CodCom**, CodPortal, NumeroP, NumViv)

Elementos_Comunes (CodElem, TipoElem, **CodPortal**, **CodCom**)

Vecino (CodV, **DNIV**, Edad, **numCuenta**, nom, ape1, ape2(*), esPropietario, fechaIni, fechaFin (*), **CodPortal**, **CodCom**)

tfnoV (numtelV, **CodV**)

Reunión (CodR, OrdenDía, Acta, Duración)

Instalación (CodI, Tipo, Estado, **CodCom**)

Necesidad (CodN, tipoN, **CodCom**)

Cotidiana (**CodN**)

Avería (**CodN**, Urgencia)

Empresa (CIF, Actividad, Tamaño, **NomCom**, DirecciónE)

presidente(**CodV**, **CodCom**, fechaE)

asiste (**CodR**, **CodV**, fechaR)

satisface (CIF, **CodN**, fechaIni, fechaFin(*), Precio)

5. Script SQL.

Código SQL: Raúl Palomo Mazo y Pablo Galarón Mateo

```
/* -----  
Raúl Palomo Mazo y Pablo Galarón Mateo  
Administración de Fincas  
----- */
```

```
/* -----  
Definición de la estructura de la base de datos  
----- */
```

```
DROP DATABASE IF EXISTS AdministracionFincas;  
CREATE DATABASE AdministracionFincas;  
USE AdministracionFincas;  
SET SQL_SAFE_UPDATES = 0;
```

```
DROP TABLE IF EXISTS Ciudad;  
DROP TABLE IF EXISTS Empresa;  
DROP TABLE IF EXISTS Sede;  
DROP TABLE IF EXISTS Personal;  
DROP TABLE IF EXISTS TelefonoPersonal;  
DROP TABLE IF EXISTS Comunidad;  
DROP TABLE IF EXISTS Portal;  
DROP TABLE IF EXISTS ElementosComunes;  
DROP TABLE IF EXISTS Vecino;  
DROP TABLE IF EXISTS TelefonoVecino;  
DROP TABLE IF EXISTS Presidente;  
DROP TABLE IF EXISTS Reunion;  
DROP TABLE IF EXISTS Asiste;  
DROP TABLE IF EXISTS Instalacion;  
DROP TABLE IF EXISTS Necesidad;  
DROP TABLE IF EXISTS Cotidiana;  
DROP TABLE IF EXISTS Averia;  
DROP TABLE IF EXISTS Satisface;
```

```
-- Tabla Ciudad (Debe existir antes de que Sede pueda referenciarla)  
CREATE TABLE Ciudad (  
    CodC INT PRIMARY KEY,  
    nombreC VARCHAR(100) NOT NULL,  
    Poblacion INT NOT NULL  
);
```


-- Tabla Empresa (No tiene dependencias)

```
CREATE TABLE Empresa (  
    CIF INT PRIMARY KEY,  
    NomComercial VARCHAR(100) NOT NULL UNIQUE,  
    DireccionE VARCHAR(200) NOT NULL,  
    Actividad VARCHAR(100) NOT NULL,  
    Tamaño VARCHAR(50) NOT NULL  
);
```

-- Tabla Sede (Depende de Ciudad)

```
CREATE TABLE Sede (  
    CodS INT PRIMARY KEY,  
    DireccionS VARCHAR(200) NOT NULL,  
    Horarios VARCHAR(100) NOT NULL,  
    CodC INT NOT NULL,  
    FOREIGN KEY (CodC) REFERENCES Ciudad(CodC)  
    ON DELETE CASCADE  
    ON UPDATE CASCADE  
);
```

-- Tabla Personal (Depende de Sede y Ciudad; incluye una relación reflexiva)

```
CREATE TABLE Personal (  
    CodP INT PRIMARY KEY,  
    DNIP VARCHAR(15) NOT NULL UNIQUE,  
    nom VARCHAR(100) NOT NULL,  
    ape1 VARCHAR(50) NOT NULL,  
    ape2 VARCHAR(50),  
    Salario INT NOT NULL,  
    especialidad VARCHAR(100) NOT NULL,  
    Email VARCHAR(100) NOT NULL,  
    fechaC DATE NOT NULL,  
    CodS INT NOT NULL,  
    CodJefe INT NOT NULL,  
    CodC INT NOT NULL,  
    FOREIGN KEY (CodS) REFERENCES Sede(CodS)  
    ON DELETE CASCADE  
    ON UPDATE CASCADE,  
    FOREIGN KEY (CodJefe) REFERENCES Personal(CodP)  
    ON DELETE CASCADE  
    ON UPDATE CASCADE,  
    FOREIGN KEY (CodC) REFERENCES Ciudad(CodC)  
    ON DELETE CASCADE  
    ON UPDATE CASCADE
```

-- Tabla TelefonoPersonal (Depende de Personal)

```
CREATE TABLE TelefonoPersonal (  
    CodP INT,  
    NumTel VARCHAR(15),  
    PRIMARY KEY (CodP, NumTel),  
    FOREIGN KEY (CodP) REFERENCES Personal(CodP)  
    ON DELETE CASCADE  
    ON UPDATE CASCADE  
);
```

-- Tabla Comunidad (Depende de Sede)

```
CREATE TABLE Comunidad (  
    CodCom INT PRIMARY KEY,  
    NomCom VARCHAR(100) NOT NULL,  
    Estatutos TEXT NOT NULL,  
    CodS INT NOT NULL,  
    fechaIni DATE NOT NULL,  
    fechaFin DATE,  
    FOREIGN KEY (CodS) REFERENCES Sede(CodS)  
    ON DELETE CASCADE  
    ON UPDATE CASCADE  
);
```

-- Tabla Portal (Depende de Comunidad)

```
CREATE TABLE Portal (  
    CodCom INT,  
    CodPortal INT,  
    NumeroP INT NOT NULL,  
    NumViv INT NOT NULL,  
    PRIMARY KEY(CodCom, CodPortal),  
    FOREIGN KEY (CodCom) REFERENCES Comunidad(CodCom)  
    ON DELETE CASCADE  
    ON UPDATE CASCADE  
);
```

-- Tabla ElementosComunes (Depende de Portal)

```
CREATE TABLE ElementosComunes (  
    CodElem INT PRIMARY KEY,  
    TipoElem VARCHAR(100) NOT NULL,  
    CodCom INT NOT NULL,  
    CodPortal INT NOT NULL,  
    FOREIGN KEY (CodCom, CodPortal) REFERENCES Portal(CodCom,  
CodPortal)  
    ON DELETE CASCADE
```

```
ON UPDATE CASCADE  
);
```

```
-- Tabla Vecino (Depende de Portal)
```

```
CREATE TABLE Vecino (  
    CodV INT PRIMARY KEY,  
    DNIV VARCHAR(15) NOT NULL UNIQUE,  
    numCuenta VARCHAR(30) NOT NULL UNIQUE,  
    nom VARCHAR(100) NOT NULL,  
    ape1 VARCHAR(50) NOT NULL,  
    ape2 VARCHAR(50),  
    edad INT CHECK (edad >= 0),  
    esPropietario BOOLEAN NOT NULL,  
    CodCom INT NOT NULL,  
    CodPortal INT NOT NULL,  
    fechaIni DATE NOT NULL,  
    fechaFin DATE,  
    FOREIGN KEY (CodCom, CodPortal) REFERENCES Portal(CodCom,  
CodPortal)  
    ON DELETE CASCADE  
    ON UPDATE CASCADE  
);
```

```
-- Tabla TelefonoVecino (Depende de Vecino)
```

```
CREATE TABLE TelefonoVecino (  
    CodV INT,  
    NumTel VARCHAR(15),  
    PRIMARY KEY (CodV, NumTel),  
    FOREIGN KEY (CodV) REFERENCES Vecino(CodV)  
    ON DELETE CASCADE  
    ON UPDATE CASCADE  
);
```

```
-- Tabla Presidente (Depende de Comunidad y Vecino)
```

```
CREATE TABLE Presidente (  
    CodCom INT,  
    CodV INT,  
    fechaE DATE,  
    PRIMARY KEY(CodCom, CodV, fechaE),  
    FOREIGN KEY (CodCom) REFERENCES Comunidad(CodCom)  
    ON DELETE CASCADE  
    ON UPDATE CASCADE,  
    FOREIGN KEY (CodV) REFERENCES Vecino(CodV)  
    ON DELETE CASCADE
```

```
    ON UPDATE CASCADE  
);
```

```
-- Tabla Reunion (Depende de Comunidad)
```

```
CREATE TABLE Reunion (  
    CodR INT PRIMARY KEY,  
    Duracion INT NOT NULL,  
    OrdenDia TEXT NOT NULL,  
    Acta TEXT NOT NULL  
);
```

```
-- Tabla Asiste (Depende de Vecino y Reunion)
```

```
CREATE TABLE Asiste (  
    CodV INT,  
    CodR INT,  
    fechaR DATE,  
    PRIMARY KEY (CodV, CodR, fechaR),  
    FOREIGN KEY (CodV) REFERENCES Vecino(CodV)  
    ON DELETE CASCADE  
    ON UPDATE CASCADE,  
    FOREIGN KEY (CodR) REFERENCES Reunion(CodR)  
    ON DELETE CASCADE  
    ON UPDATE CASCADE  
);
```

```
-- Tabla Instalacion (Depende de Comunidad)
```

```
CREATE TABLE Instalacion (  
    CodI INT PRIMARY KEY,  
    Tipo VARCHAR(100) NOT NULL,  
    Estado VARCHAR(50) NOT NULL CHECK (Estado IN ('Bueno', 'Regular',  
'Malo')),  
    CodCom INT NOT NULL,  
    FOREIGN KEY (CodCom) REFERENCES Comunidad(CodCom)  
    ON DELETE CASCADE  
    ON UPDATE CASCADE  
);
```

```
-- Tabla Necesidad (Depende de Comunidad)
```

```
CREATE TABLE Necesidad (  
    CodN INT PRIMARY KEY,  
    tipoN VARCHAR(100) NOT NULL,  
    CodCom INT NOT NULL,  
    FOREIGN KEY (CodCom) REFERENCES Comunidad(CodCom)  
    ON DELETE CASCADE
```

```
ON UPDATE CASCADE  
);
```

-- Subtipo Cotidiana, Avería y Urgencia (Dependen de Necesidad)

```
CREATE TABLE Cotidiana (  
    CodN INT PRIMARY KEY,  
    FOREIGN KEY (CodN) REFERENCES Necesidad(CodN)  
    ON DELETE CASCADE  
    ON UPDATE CASCADE  
);
```

```
CREATE TABLE Averia (  
    CodN INT PRIMARY KEY,  
    GradoUrgencia INT NOT NULL CHECK (GradoUrgencia BETWEEN 1 AND  
5),  
    FOREIGN KEY (CodN) REFERENCES Necesidad(CodN)  
    ON DELETE CASCADE  
    ON UPDATE CASCADE  
);
```

-- Tabla Satisface (Depende de Empresa y Necesidad)

```
CREATE TABLE Satisface (  
    CIF INT,  
    CodN INT,  
    fechaIni DATE NOT NULL,  
    fechaFin DATE,  
    Precio INT,  
    PRIMARY KEY (CIF, CodN),  
    FOREIGN KEY (CIF) REFERENCES Empresa(CIF)  
    ON DELETE CASCADE  
    ON UPDATE CASCADE,  
    FOREIGN KEY (CodN) REFERENCES Necesidad(CodN)  
    ON DELETE CASCADE  
    ON UPDATE CASCADE  
);
```

```

/*-----
Trigger de inserción
-----*/

```

#1- Comprobar que los vecinos que asisten a una reunión son de la misma comunidad.

```

delimiter //
create trigger AsistenciaReunionBI before insert
on asiste
for each row
begin
-- contamos cuantos vecinos asisten a la misma reunión(tienen que ser de la
misma comunidad)
    declare numVecinosAntiguos int;
    set numVecinosAntiguos = (select count(CodV)
                             from asiste
                             where CodR = new.CodR
                             group by CodR);
-- Comprobamos que al menos hay un asistente en la reunión(si es 0
insertamos sin problema)
    if numVecinosAntiguos >= 1 then
-- Comprobamos que CodCom de los vecinos que están en la reunión es el
mismo que el del vecino que queremos insertar
        if (select CodCom
            from vecino
            where CodV = (select CodV
                          from asiste
                          where CodR = new.CodR -- CodCom de los vecinos que ya
están insertados de antes
                          limit 1)) <> (select CodCom
                                      from vecino
                                      where CodV = new.CodV) then -- CodCom del que
queremos insertar
            signal sqlstate '45000' set message_text = 'El vecino no
pertenece a la comunidad de la reunión a la que quiere asistir';
        end if;
    end if;
end;
//
delimiter ;

```

#2- Comprobar que a la hora de insertar un presidente, no exista otro vecino en esa comunidad que ya esté ocupando el cargo.

```
delimiter //
create trigger NuevoPresidenteBI before insert
on presidente
for each row
begin
-- fecha en la que fue elegido el último presidente para después comparar
declare ultimaFecha date;
set ultimaFecha = (select max(fechaE)
                    from presidente
                    where CodCom = new.CodCom
                    group by CodCom);
-- si han pasado menos de 4 años(1461 días) todavía no puede ser
presidente porque el actual no acabó su mandato
if datediff(new.fechaE, ultimaFecha) < 1461 then
    signal sqlstate '45000' set message_text = 'El actual presidente de
la comunidad no ha acabado su mandato';
end if;
end;
//
delimiter ;
```

#3- Comprobar que el presidente elegido pertenece a la comunidad en la que va a ser presidente.

```
delimiter //
create trigger PresiComBI before insert
on presidente
for each row
begin
-- obtener la comunidad a la que pertenece el presidente que estamos
insertando
declare CodComPresi int;
set CodComPresi = (select CodCom
                    from vecino
                    where CodV = new.CodV);
-- comprobar si el CodCom del presidente que insertamos es el mismo
que el de la comunidad a la que pertenece y va a ser presidente
if CodComPresi <> new.CodCom then
    signal sqlstate '45000' set message_text = 'Este vecino no puede
ser presidente de una comunidad a la que no pertenece';
end if;
end;
```

end;

//

delimiter ;

/*-----

Inserción de datos

-----*/

INSERT INTO Ciudad (CodC, nombreC, Poblacion) VALUES

(1, 'Madrid', 3200000),
(2, 'Barcelona', 1600000),
(3, 'Valencia', 800000),
(4, 'Sevilla', 700000),
(5, 'Zaragoza', 675000),
(6, 'Málaga', 580000),
(7, 'Murcia', 460000),
(8, 'Bilbao', 345000),
(9, 'Alicante', 330000),
(10, 'Córdoba', 325000);

INSERT INTO Empresa (CIF, NomComercial, DireccionE, Actividad, Tamaño)
VALUES

(101, 'Mantenimiento Total', 'Calle Mayor 1, Madrid', 'Mantenimiento de Edificios', 'Grande'),
(102, 'Agua Viva', 'Av. Diagonal 230, Barcelona', 'Mantenimiento de Piscinas', 'Mediana'),
(103, 'Verde Urbano', 'Calle Larga 15, Valencia', 'Jardineria', 'Pequeña'),
(104, 'Limpieza Integral', 'Calle de la Torre 20, Sevilla', 'Limpieza', 'Mediana'),
(105, 'Cerrajeros ZGZ', 'Av. Central 45, Zaragoza', 'Cerrajería y Puertas', 'Pequeña'),
(106, 'Fit and Company', 'Calle Alta 9, Málaga', 'Gimnasios', 'Mediana'),
(107, 'Pintores Hernandez', 'Calle del Sol 12, Murcia', 'Pintura', 'Pequeña'),
(108, 'EcoEmbes', 'Plaza del Mercado 7, Bilbao', 'Reciclaje', 'Grande'),
(109, 'Tejados Marín', 'Av. Costa Blanca 32, Alicante', 'Reparaciones', 'Mediana'),
(110, 'Securitas Direct', 'Calle Ancha 21, Córdoba', 'Sistemas de Seguridad', 'Pequeña');

INSERT INTO Sede (CodS, DireccionS, Horarios, CodC) VALUES

(1, 'Calle Gran Vía 12, Madrid', '9:00-18:00', 1),
(2, 'Av. Meridiana 80, Barcelona', '9:00-17:30', 2),
(3, 'Calle Serranos 15, Valencia', '8:00-16:00', 3),
(4, 'Plaza Nueva 8, Sevilla', '9:00-18:00', 4),
(5, 'Paseo Independencia 45, Zaragoza', '8:30-16:30', 5),

(6, 'Av. Andalucía 22, Málaga', '10:00-19:00', 6),
 (7, 'Calle Ronda 5, Murcia', '9:00-17:00', 7),
 (8, 'Calle Ercilla 14, Bilbao', '8:00-15:00', 8),
 (9, 'Plaza Luceros 1, Alicante', '9:00-17:30', 9),
 (10, 'Calle Cruz Conde 11, Córdoba', '8:30-16:30', 10);

INSERT INTO Personal (CodP, nom, DNIP, ape1, ape2, especialidad, Email, fechaC, CodS, CodJefe, CodC, Salario) VALUES

(1, 'Ana', '12345678A', 'Pérez', 'Gómez', 'Administración',
 'ana.perez@adminfincas.com', '2020-01-15', 1, 1, 1, 2000),
 (2, 'Luis', '87654321B', 'López', 'Martín', 'Mantenimiento',
 'luis.lopez@adminfincas.com', '2019-06-10', 2, 1, 2, 1500),
 (3, 'María', '45612378C', 'García', 'Fernández', 'Finanzas',
 'maria.garcia@adminfincas.com', '2018-04-05', 3, 1, 3, 1875),
 (4, 'Juan', '65478912D', 'Rodríguez', 'Santos', 'Atención al Cliente',
 'juan.rodriguez@adminfincas.com', '2021-09-12', 4, 3, 4, 1589),
 (5, 'Carmen', '98765432E', 'Hernández', 'Ruiz', 'Técnico de Sistemas',
 'carmen.hernandez@adminfincas.com', '2017-03-22', 5, 3, 5, 2014),
 (6, 'Miguel', '32165487F', 'Sánchez', 'López', 'Limpieza',
 'miguel.sanchez@adminfincas.com', '2016-08-18', 6, 5, 6, 1254),
 (7, 'Rosa', '78912345G', 'Martínez', 'Díaz', 'Jardinería',
 'rosa.martinez@adminfincas.com', '2022-02-28', 7, 5, 7, 1258),
 (8, 'Carlos', '65432198H', 'Jiménez', 'Morales', 'Reformas',
 'carlos.jimenez@adminfincas.com', '2023-05-10', 8, 6, 8, 2000),
 (9, 'Laura', '32198765I', 'González', 'Pérez', 'Contabilidad',
 'laura.gonzalez@adminfincas.com', '2021-11-30', 9, 7, 9, 1000),
 (10, 'Pedro', '98732145J', 'Fernández', 'Núñez', 'Gestión de Reuniones',
 'pedro.fernandez@adminfincas.com', '2018-07-15', 10, 8, 10, 4000),
 (11, 'Alberto', '98765431K', 'Molina', 'Fernández', 'Administración',
 'alberto.molina@adminfincas.com', '2022-04-10', 1, 1, 1, 1258),
 (12, 'Lucía', '87654321L', 'Gómez', 'Santos', 'Mantenimiento',
 'lucia.gomez@adminfincas.com', '2023-01-15', 2, 2, 2, 3000),
 (13, 'Raúl', '12345679M', 'Pérez', 'Jiménez', 'Finanzas',
 'raul.perez@adminfincas.com', '2019-07-20', 3, 3, 3, 1000),
 (14, 'Sara', '65478911N', 'Hernández', 'López', 'Atención al Cliente',
 'sara.hernandez@adminfincas.com', '2020-10-25', 4, 4, 4, 1256),
 (15, 'David', '98765432O', 'López', 'Martínez', 'Técnico de Sistemas',
 'david.lopez@adminfincas.com', '2018-06-05', 5, 5, 5, 2587),
 (16, 'Elena', '32165487P', 'Martín', 'Díaz', 'Limpieza',
 'elena.martin@adminfincas.com', '2021-03-18', 6, 6, 6, 1000),
 (17, 'Pablo', '78912345Q', 'Ruiz', 'García', 'Jardinería',
 'pablo.ruiz@adminfincas.com', '2020-09-12', 7, 7, 7, 2000),
 (18, 'Claudia', '65432198R', 'Santos', 'Moreno', 'Reformas',
 'claudia.santos@adminfincas.com', '2022-12-30', 8, 8, 8, 1452),

```
(19, 'Hugo', '32198765S', 'Jiménez', 'Pérez', 'Contabilidad',
'hugo.jimenez@adminfincas.com', '2019-05-25', 9, 9, 9, 2522),
(20, 'Julia', '98732145T', 'González', 'Morales', 'Gestión de Reuniones',
'julia.gonzalez@adminfincas.com', '2020-11-15', 10, 10, 10, 1000),
(21, 'Sergio', '12345000U', 'Vargas', 'Fernández', 'Administración',
'sergio.vargas@adminfincas.com', '2021-08-01', 1, 1, 1, 2414),
(22, 'Nerea', '65478912V', 'Romero', 'Pérez', 'Atención al Cliente',
'nerea.romero@adminfincas.com', '2022-02-10', 2, 2, 2, 1200),
(23, 'Marcos', '32165400W', 'Blanco', 'López', 'Mantenimiento',
'marcos.blanco@adminfincas.com', '2018-11-12', 3, 3, 3, 1456),
(24, 'Isabel', '78912346X', 'Cano', 'Martínez', 'Técnico de Sistemas',
'isabel.cano@adminfincas.com', '2017-09-07', 4, 4, 4, 2000),
(25, 'Javier', '98765433Y', 'Díaz', 'Ruiz', 'Contabilidad',
'javier.diaz@adminfincas.com', '2021-01-23', 5, 5, 5, 1400),
(26, 'Paula', '32165489Z', 'Rojas', 'Morales', 'Limpieza',
'paula.rojas@adminfincas.com', '2023-06-15', 6, 6, 6, 3000),
(27, 'Ángel', '78912347A', 'Vega', 'Gómez', 'Reformas',
'angel.vega@adminfincas.com', '2020-07-19', 7, 7, 7, 4700),
(28, 'Natalia', '65432199B', 'Serrano', 'Núñez', 'Finanzas',
'natalia.serrano@adminfincas.com', '2019-03-22', 8, 8, 8, 1000),
(29, 'Adrián', '32198766C', 'Ortiz', 'Santos', 'Gestión de Reuniones',
'adrian.ortiz@adminfincas.com', '2020-04-30', 9, 9, 9, 5000),
(30, 'Alicia', '98732146D', 'Medina', 'López', 'Jardinería',
'alicia.medina@adminfincas.com', '2022-09-18', 10, 10, 10, 1200);
```

```
INSERT INTO TelefonoPersonal (CodP, NumTel) VALUES
```

```
(1, '600123456'),
(2, '600234567'),
(3, '600345678'),
(4, '600456789'),
(5, '600567890'),
(6, '600678901'),
(7, '600789012'),
(8, '600890123'),
(9, '600901234'),
(10, '600012345');
```

```
INSERT INTO Comunidad (CodCom, NomCom, Estatutos, CodS, fechaIni,
fechaFin) VALUES
```

```
(1, 'Residencial Gran Vía', 'Normas básicas de convivencia.', 1, '2020-01-01',
NULL),
(2, 'Edificio Diagonal', 'Reglas para zonas comunes.', 2, '2021-02-01', '2024-01-
29'),
(3, 'Barrio Serranos', 'Normativa actualizada en 2023.', 3, '2019-05-01', NULL),
```

```
(4, 'Conjunto Nueva Plaza', 'Estatutos registrados en 2020.', 4, '2020-07-15',
NULL),
(5, 'Complejo Zaragoza', 'Normativa adaptada en 2021.', 5, '2018-09-20',
NULL),
(6, 'Zona Andalucía', 'Actualización pendiente.', 6, '2018-06-12', NULL),
(7, 'Residencial Sol', 'Estatutos desde 2019.', 7, '2015-03-05', NULL),
(8, 'Edificio Ercilla', 'Normas internas para vecinos.', 8, '2013-01-10', '2024-12-
07'),
(9, 'Paseo Luceros', 'Estatutos ratificados en 2020.', 9, '2011-11-22', NULL),
(10, 'Residencial Cruz Conde', 'Aprobados en 2017.', 10, '2017-08-01', NULL);
```

```
INSERT INTO Portal (CodCom, CodPortal, NumeroP, NumViv) VALUES
```

```
(1, 1, 101, 8),
(1, 2, 102, 10),
(2, 1, 201, 12),
(3, 1, 301, 15),
(4, 1, 401, 20),
(5, 1, 501, 6),
(6, 1, 601, 9),
(7, 1, 701, 4),
(8, 1, 801, 11),
(9, 1, 901, 5);
```

```
INSERT INTO ElementosComunes (CodElem, TipoElem, CodCom, CodPortal)
VALUES
```

```
(1, 'Ascensor', 1, 2),
(2, 'Buzon', 2, 1),
(3, 'Cuarto de basuras', 9, 1),
(4, 'Extintor', 3, 1);
```

```
INSERT INTO Vecino (CodV, DNIV, numCuenta, nom, ape1, ape2, edad,
esPropietario, CodCom, CodPortal, fechaIni, fechaFin) VALUES
```

```
(1, '11223344A', 'ES12345678901234567890', 'Carlos', 'López', 'Martínez', 35,
TRUE, 1, 1, '2020-01-01', NULL),
(2, '22334455B', 'ES98765432109876543210', 'María', 'Gómez', 'Fernández',
29, FALSE, 1, 2, '2021-06-15', NULL),
(3, '33445566C', 'ES11112222333344445555', 'José', 'Pérez', 'Rodríguez', 40,
TRUE, 2, 1, '2019-03-10', NULL),
(4, '44556677D', 'ES55554444333322221111', 'Ana', 'Santos', 'Hernández', 50,
TRUE, 3, 1, '2018-12-01', NULL),
(5, '55667788E', 'ES44445555666677778888', 'Rosa', 'Jiménez', 'Ruiz', 45,
TRUE, 4, 1, '2017-07-20', NULL),
(6, '66778899F', 'ES88887777666655554444', 'Luis', 'García', 'Díaz', 38, TRUE,
5, 1, '2018-02-28', NULL),
```

```
(7, '77889900G', 'ES22223333444455556666', 'Laura', 'Hernández', 'López', 25,
FALSE, 6, 1, '2016-03-15', NULL),
(8, '88990011H', 'ES99990000111122223333', 'Juan', 'Núñez', 'Morales', 60,
TRUE, 7, 1, '2015-05-25', '2021-09-09'),
(9, '99001122I', 'ES33334444555566667777', 'Carmen', 'Luna', 'Martínez', 42,
TRUE, 8, 1, '2014-11-12', NULL),
(10, '00112233J', 'ES11112222333344445555', 'Miguel', 'Duarte', 'Santos', 28,
FALSE, 9, 1, '2014-09-07', NULL),
(11, '11224455K', 'ES12312312312312312345', 'Roberto', 'Gómez', 'Pérez', 32,
TRUE, 1, 2, '2024-01-01', NULL),
(12, '22335566L', 'ES98798798798798798765', 'Lucía', 'Martínez', 'López', 26,
FALSE, 2, 1, '2024-01-01', NULL),
(13, '33446677M', 'ES11114444433322211111', 'Diego', 'Santos', 'Jiménez', 37,
TRUE, 3, 1, '2024-01-01', NULL),
(14, '44557788N', 'ES55556666777788889999', 'Paula', 'Ruiz', 'García', 44,
FALSE, 4, 1, '2024-01-01', NULL),
(15, '55668899O', 'ES44445555666677778889', 'Adrián', 'López', 'Hernández',
39, TRUE, 5, 1, '2024-01-01', NULL),
(16, '66779900P', 'ES88889999111122223333', 'Clara', 'Morales', 'Pérez', 22,
FALSE, 6, 1, '2024-01-01', NULL),
(17, '77880011Q', 'ES22224444666677778888', 'Iván', 'Núñez', 'Martínez', 58,
TRUE, 7, 1, '2024-01-01', NULL),
(18, '88991122R', 'ES99998888777766665555', 'Sofía', 'Luna', 'Santos', 30,
FALSE, 8, 1, '2024-01-01', NULL),
(19, '99002233S', 'ES33334444555566667888', 'Pedro', 'Duarte', 'Ruiz', 45,
TRUE, 9, 1, '2024-01-01', NULL);
```

INSERT INTO TelefonoVecino (CodV, NumTel) VALUES

```
(1, '650123456'),
(2, '650234567'),
(3, '650345678'),
(4, '650456789'),
(5, '650567890'),
(6, '650678901'),
(7, '650789012'),
(8, '650890123'),
(9, '650901234'),
(10, '650012345');
```

INSERT INTO Presidente (CodCom, CodV, fechaE) VALUES

```
(1, 1, '2023-01-01'),
(2, 3, '2022-07-01'),
(3, 4, '2021-09-15'),
(4, 5, '2020-11-20'),
```

```
(5, 6, '2019-05-05'),  
(6, 7, '2018-02-10'),  
(7, 8, '2017-12-25'),  
(8, 9, '2016-06-30'),  
(9, 10, '2015-03-01'),  
(9, 10, '2019-03-01');
```

```
INSERT INTO Reunion (CodR, Duracion, OrdenDia, Acta) VALUES  
(1, 90, 'Presupuesto anual y mejoras', 'Aprobado por unanimidad'),  
(2, 120, 'Cambio de presidente y cuentas', 'Debate y aprobación'),  
(3, 60, 'Incidencias y nuevos contratos', 'Pendiente de firma'),  
(4, 45, 'Limpieza y mantenimiento', 'Aprobación del plan de acción'),  
(5, 100, 'Propuestas de vecinos', 'Votación y acta registrada'),  
(6, 80, 'Revisión de estatutos', 'Rechazado por mayoría'),  
(7, 70, 'Problemas de convivencia', 'Mediación aceptada'),  
(8, 50, 'Reparaciones urgentes', 'Informe aprobado'),  
(9, 110, 'Cierre de cuentas anuales', 'Cierre sin incidencias'),  
(10, 40, 'Proyectos de futuro', 'Pendiente para próxima reunión'),  
(11, 15, 'Excrementos en el patio', 'Represalias correspondientes'),  
(12, 30, 'Apertura piscina', 'Aprobación fecha');
```

```
INSERT INTO Asiste (CodV, CodR, fechaR) VALUES  
(1, 1, '2023-02-01'),  
(2, 1, '2023-02-01'),  
(3, 2, '2023-03-10'),  
(4, 3, '2023-03-10'),  
(5, 4, '2023-04-15'),  
(6, 5, '2023-05-20'),  
(7, 6, '2023-06-25'),  
(8, 7, '2023-07-30'),  
(9, 8, '2023-08-15'),  
(10, 9, '2023-09-10'),  
(2, 11, '2023-01-04'),  
(2, 12, '2023-07-15'),  
(1, 11, '2023-01-04');
```

```
INSERT INTO Instalacion (CodI, Tipo, Estado, CodCom) VALUES  
(1, 'Piscina', 'Bueno', 1),  
(2, 'Zona infantil', 'Regular', 2),  
(3, 'Parking', 'Malo', 3),  
(4, 'Jardín', 'Bueno', 4),  
(5, 'Cafeteria', 'Bueno', 5),  
(6, 'Gimnasio', 'Regular', 6),  
(7, 'Terraza', 'Bueno', 8),
```

(8, 'Área deportiva', 'Regular', 9),
(9, 'Barbacoa', 'Bueno', 10),
(10, 'Barbacoa', 'Regular', 1),
(11, 'Terraza', 'Bueno', 1),
(12, 'Área deportiva', 'Regular', 1),
(13, 'Barbacoa', 'Bueno', 1);

INSERT INTO Necesidad (CodN, tipoN, CodCom) VALUES

(1, 'Reparación de ascensor', 1),
(2, 'Mantenimiento de piscina', 2),
(3, 'Limpieza de jardín', 3),
(4, 'Limpieza Portales', 4),
(5, 'Sustitución de puerta', 5),
(6, 'Reparación de gimnasio', 6),
(7, 'Reparación telefonillo', 1),
(8, 'Recogida de basuras', 8),
(9, 'Reparación de tejado', 9),
(10, 'Instalación de cámaras', 10);

INSERT INTO Cotidiana (CodN) VALUES

(2), (3), (4), (8);

INSERT INTO Averia (CodN, GradoUrgencia) VALUES

(1, 5), (5, 3), (6, 2), (7, 1), (9, 4), (10, 3);

INSERT INTO Satisface (CIF, CodN, fechaIni, fechaFin, Precio) VALUES

(101, 1, '2022-02-01', '2022-02-10', 1000),
(102, 2, '2023-03-01', NULL, NULL),
(103, 3, '2021-04-01', NULL, NULL),
(104, 4, '2019-05-01', NULL, NULL),
(105, 5, '2020-06-01', '2020-06-25', 500),
(106, 6, '2021-07-01', '2021-07-20', 320),
(101, 7, '2023-08-01', '2023-08-15', 50),
(108, 8, '2023-09-01', NULL, NULL),
(109, 9, '2023-10-01', '2023-10-15', 600),
(110, 10, '2015-11-01', '2015-11-20', 250);

```

/*-----
Otros trigger
-----*/

```

#4- Si un vecino deja de ser propietario, actualizar automáticamente su fecha de baja.

-- Es decir, cuando hagamos un update y el campo pase de 1 a 0 poner automáticamente la fecha en la que se ha hecho es baja que
 -- sería la fechaFin

```

delimiter //
create trigger FinPropietarioBU before update
on vecino
for each row
begin
    if new.esPropietario = 0 and old.esPropietario = 1 then
        set new.fechaFin = curdate();
    end if;
end;
//
delimiter ;

```

#5- Guardar el número de teléfono y información de los trabajadores que se eliminan de la tabla (despedidos) por si hiciera falta contactar con ellos en el futuro.

```

CREATE TABLE InfoDespedidos(
    numTel VARCHAR(15),
    nombrePersonal VARCHAR(50),
    fecDespido DATE,
    usuario VARCHAR(50)
)
;

delimiter //
create trigger InfoDespedidosBD before delete
on personal
for each row
begin
    declare numTel varchar(15);
    set numTel = (select NumTel
                  from telefonopersonal
                  where CodP = old.CodP);

```

```
insert into InfoDespedidos
    values (numTel, concat(old.nom, ' ', old.ape1, ' ', old.ape2),
curdate(), user());
end;
//
delimiter ;

/*-----
Consultas, modificaciones, borrados y vistas con enunciado
-----*/
```

#1 Nombre de las comunidades que tiene más averías que la media.

-- creamos una vista para saber el número de averías que ha tenido cada comunidad.

```
create view NumAveriasTotal as
    select c.NomCom, count(a.CodN) as NumAverias
    from comunidad as c inner join necesidad as n on c.CodCom =
n.CodCom
    inner join averia as a on n.CodN = a.CodN
    group by c.CodCom;
```

-- seleccionamos las comunidades que tengan averías por encima de la media

```
select NomCom, NumAverias
from NumAveriasTotal
where NumAverias > (select avg(NumAverias)
                    from NumAveriasTotal);
```

#2 Nombre del vecino que ha asistido a más reuniones.

```
create view reunionesVecino as
    select concat(nom, ' ', ape1, ' ', ape2) as nombreV, count(fechaR) as
numReuniones
    from asiste as a inner join vecino as v on a.CodV = v.CodV
    group by a.CodV;
```

```
select nombreV, numReuniones
from reunionesVecino
where numReuniones = (select max(numReuniones)
                    from reunionesVecino);
```

#3 Nombre de las comunidades que han dejado de ser administradas por la sede en el último año.

```
select nomCom, fechaIni, fechaFin
from comunidad
where year(fechaFin) > (year(curdate()) - 1);
```


#4 Necesidades cotidianas que han surgido en cada una de las comunidades y empresa que las ha solucionado.

```
select nomCom, tipoN, NomComercial
from comunidad as com inner join necesidad as n on com.codCom = n.codCom
    inner join cotidiana as c on n.codN = c.codN
    inner join satisface as s on c.codN = s.codN
    inner join empresa as e on s.CIF = e.CIF;
```

#5 Comprueba si ha habido algún presidente que haya abandonado la comunidad después de su cargo. ¿Quién?

```
select concat(nom, ' ', ape1, ' ', ape2) nombreV, fechaE, fechaFin
from vecino as v inner join presidente as p on v.codV = p.codV
where fechaFin > fechaE;
```

#6 Comprueba si ha habido algún vecino que haya sido elegido presidente más de una vez en alguna comunidad. En ese caso, haya su nombre, la fecha de las diferentes elecciones en las que ha sido elegido y la comunidad.

```
create view presidenteVariasVeces as
select p.codV, count(p.codV) as numVeces
from vecino as v inner join presidente as p on v.codV = p.codV
group by p.codV
having numVeces > 1;
```

```
select concat(nom, ' ', ape1, ' ', ape2) as nombreV, fechaE, nomCom
from vecino as v inner join presidente as p on v.codV = p.codV
    inner join comunidad as c on p.Codcom = c.CodCom
where p.codV = (select codV
                from presidenteVariasVeces);
```

#7 Determina que avería ha sido la que más ha tardado en solucionarse. Nos interesa saber el grado de urgencia de esta y la empresa encargada de su reparación, para comprobar si dicha empresa ha sido efectiva.

-- Haz lo mismo para la avería que se ha solucionado más rápido

```
drop view if exists tiempoReparacion;
create view tiempoReparacion as
select datediff(fechaFin, fechaIni) diasReparacion
from satisface;
-- Avería que más se ha tardado en solucionar
select tipoN averia, datediff(fechaFin, fechaIni) diasReparacion, gradoUrgencia,
nomComercial empresa
from necesidad n inner join averia a on n.codN = a.codN
    inner join satisface s on a.codN = s.codN
```

```

inner join empresa e on e.CIF = s.CIF
where datediff(fechaFin, fechaIni) = (select max(diasReparacion) from
tiempoReparacion);
-- Avería solucionada más rápido
select tipoN averia, datediff(fechaFin, fechaIni) diasReparacion, gradoUrgencia,
nomComercial empresa
from necesidad n inner join averia a on n.codN = a.codN
inner join satisface s on a.codN = s.codN
inner join empresa e on e.CIF = s.CIF
where datediff(fechaFin, fechaIni) = (select min(diasReparacion) from
tiempoReparacion);

```

#8 Para cada comunidad, halla el número del portal que tenga ascensor pero no disponga de cuarto de basuras.

```

select p.CodCom, NumeroP, TipoElem
from portal as p inner join elementoscomunes as e on p.CodCom = e.CodCom
where p.CodCom in (select e.CodCom
from elementoscomunes as e inner join portal as p
on e.CodCom = p.CodCom
where TipoElem like 'Ascensor') and p.CodCom not in (select
e.CodCom
from elementoscomunes as e inner join
portal as p on e.CodCom = p.CodCom
where TipoElem like 'Cuarto de
basuras');

```

#9 Aumentar el salario de todo el personal en un 5% que lleve trabajando más de 5 años en la empresa.

```

set sql_safe_updates=0;
update personal
set Salario = Salario * 1.05
where datediff(curdate(), fechaC) > (365 * 5 + 1);

```

#10 Datos de la comunidad que más dinero ha hecho ganar a empresas de mantenimiento.

```

select c.CodCom, NomCom
from comunidad as c inner join necesidad as n on c.CodCom = n.CodCom
inner join averia as a on a.CodN = n.CodN
inner join satisface as s on s.CodN = a.CodN
inner join empresa as e on e.CIF = s.CIF
where Actividad like '%Mantenimiento%'
group by c.CodCom
having sum(precio) = (select sum(precio) as total

```

```

as e on s.CIF = e.CIF
                                from satisface as s inner join empresa
                                group by e.CIF
                                order by total desc
                                limit 1);

```

#11 Eliminar las averías con grado de urgencia mínimo.

```

delete from necesidad
where CodN in (select CodN
               from averia
               where GradoUrgencia = 1);

```

#12 Borrar los vecinos cuyo nombre empiece por la letra 'R', su número de teléfono acabe en 0 y hayan asistido a una sola reunión.

-- Rosa, está comprobado.

```

delete from vecino
where nom like 'R%' and CodV in (select CodV
from telefonovecino
where NumTel like '%0') and CodV in (select CodV
from asiste
group by CodV
having count(CodR) = 1);

```

#13 Realiza una comparativa de los salarios del personal en función de la ciudad.

```

select nombreC, avg(Salario) as SalarioPromedio, max(Salario) as
SalarioMaximo, min(Salario) as SalarioMinimo
from personal as p inner join ciudad as c on c.CodC = p.CodC
group by c.CodC
order by SalarioPromedio desc;

```

#14 Comunidades con instalaciones en peor estado.

```

select nomCom, tipo, estado
from instalacion as i inner join comunidad as c on i.codCom = c.codCom
where estado like 'Malo';

```

#15 Obtén el total de empleados por sede.

```

select DireccionS, count(*) as TotalEmpleados
from personal as p inner join sede as s on p.CodS = s.CodS
group by DireccionS;

```

#16 Cuenta la cantidad de instalaciones por tipo en la comunidad "Residencial Gran Vía".

```
select Tipo, count(*) as Cantidad
from instalacion as i inner join comunidad as c on i.CodCom = c.CodCom
where c.NomCom = 'Residencial Gran Vía'
group by Tipo;
```

6. Conclusión.

En conclusión, el desarrollo de esta base de datos para la administración de fincas ha permitido crear una estructura robusta y eficiente para gestionar la información relevante de las sedes, comunidades, portales, vecinos y empresas externas. A través del modelo relacional implementado en SQL y la inclusión de triggers, se ha logrado garantizar la integridad de los datos y facilitar la automatización de procesos, lo que mejora la eficiencia operativa de la empresa de administración de fincas.

Las consultas SQL desarrolladas han demostrado ser herramientas útiles para extraer información relevante y apoyar la toma de decisiones en aspectos clave, como la gestión de instalaciones, necesidades de mantenimiento y la organización de reuniones comunitarias. Además, el sistema propuesto permite un seguimiento detallado de los trabajadores, vecinos y sus interacciones dentro de las comunidades, lo que facilita la administración y mejora la calidad del servicio proporcionado.

Este trabajo destaca la importancia de un diseño adecuado de bases de datos para el manejo de grandes volúmenes de información en contextos complejos, como el de la administración de fincas. Con una base de datos bien estructurada, se optimizan tanto los recursos internos de la empresa como las relaciones con las comunidades y las distintas empresas con las que trabaja, mejorando así la gestión integral de los servicios y las necesidades de los clientes.